Project 1: Local Data Engineering Pipeline - Learning & Debugging Report

Abstract

This project focused on building a complete local data engineering pipeline using Python, Polars, DuckDB, GitHub, and SQL. The objective was to understand the end-to-end flow of data from ingestion to analytics while gaining practical experience with debugging, version control, and local development environments.

Project Architecture

Extract → Transform → Load → Query → Insights

Extract: Read raw CSV data.
Transform: Clean and enrich data using Polars.
Load: Store transformed data in DuckDB.
Query: Run SQL analytics queries.
Insights: Generate business-focused outputs.

Technologies Used

Python
Polars
DuckDB
SQL
Git & GitHub
Codex
Terminal (macOS)
Nano Editor

Major Learnings

Understanding ETL pipelines.
Working with local databases.
Writing SQL analytics queries.
Using Git branches, commits, pull requests, and merges.
Reading Python stack traces.
Using grep to search code.
Editing files directly from the terminal.

Debugging Journey

1. Repository Issues
- Cloned repositories and encountered directory/path issues.

2. Environment Issues
- Missing Polars package.

- Missing DuckDB package.
- Installed dependencies and verified environment setup.

3. Database Issues
- DuckDB reported that the employees table did not exist.
- Investigation revealed that only the sales table had been loaded.

4. Git & Merge Conflict Issues
- Pull requests introduced merge conflicts.
- Both old and new query logic were accidentally retained.
- Learned how merge conflicts occur and why careful resolution matters.

5. File Version Confusion
- The code being executed was not always the same code being viewed.
- Learned to verify the actual local file using grep and direct inspection.

6. Indentation Errors
- Python raised IndentationError due to inconsistent spacing.
- Learned that Python structure is controlled by indentation.

7. Root Cause Discovery
- The actual issue was not DuckDB or SQL.
- The local src/04_query.py file still contained legacy employees queries.
- Once removed and replaced with sales-based queries, the pipeline executed successfully.

Key Commands Learned

python run_pipeline.py
python src/04_query.py
grep -n "employees" src/04_query.py
nano src/04_query.py
pip install -r requirements.txt
git clone
git pull

Final Outcome

Successfully built and executed a local data engineering pipeline.
Successfully queried DuckDB using SQL.
Successfully debugged package, Git, SQL, and Python issues.
Completed Phase 1 and established a foundation for future AWS, orchestration, and modern cloud data engineering work.

Personal Takeaway

The most important lesson was that debugging is often about identifying the real problem rather than immediately attempting fixes. Error messages provided the truth, and systematic investigation ultimately led to success. This project provided practical experience with real-world engineering workflows and troubleshooting.

```
(base) Harkirats-MacBook-Pro-3:project1_p1_de-main yoharks$ python run_pipeline.py
🚀 Starting local ETL pipeline

▶️ Stage 1/4: Ingest raw data
   Running: python src/01_ingest.py
✅ Wrote raw data to data/raw/sales_raw.csv
   order_id customer region  amount
0         1    Alice  North   120.0
1         2      Bob  South    75.5
2         3    Alice  North   210.0
3         4    Diana   East    50.0
4         5     Evan   West   180.0
✅ Finished stage 1/4: Ingest raw data

▶️ Stage 2/4: Transform data
   Running: python src/02_transform.py
✅ Wrote processed data to data/processed/sales_processed.csv
shape: (5, 6)
┌──────────┬──────────┬────────┬────────┬─────────────────┬───────────────┐
│ order_id ┆ customer ┆ region ┆ amount ┆ amount_with_tax ┆ amount_bucket │
│ ---      ┆ ---      ┆ ---    ┆ ---    ┆ ---             ┆ ---           │
│ i64      ┆ str      ┆ str    ┆ f64    ┆ f64             ┆ str           │
╞══════════╪══════════╪════════╪════════╪═════════════════╪═══════════════╡
│ 1        ┆ Alice    ┆ North  ┆ 120.0  ┆ 132.0           ┆ Standard      │
│ 2        ┆ Bob      ┆ South  ┆ 75.5   ┆ 83.05           ┆ Standard      │
│ 3        ┆ Alice    ┆ North  ┆ 210.0  ┆ 231.0           ┆ Large         │
│ 4        ┆ Diana    ┆ East   ┆ 50.0   ┆ 55.0            ┆ Standard      │
│ 5        ┆ Evan     ┆ West   ┆ 180.0  ┆ 198.0           ┆ Large         │
└──────────┴──────────┴────────┴────────┴─────────────────┴───────────────┘
✅ Finished stage 2/4: Transform data

▶️ Stage 3/4: Load data into DuckDB
```

```
▶️ Stage 3/4: Load data into DuckDB
   Running: python src/03_load_duckdb.py
✅ Loaded 5 rows into DuckDB table: sales
✅ Database file: db/analytics.duckdb
✅ Finished stage 3/4: Load data into DuckDB

▶️ Stage 4/4: Query DuckDB database
   Running: python src/04_query.py

✅ Total sales by region
  region  total_amount
0  North         330.0
1   West         180.0
2  South          75.5
3   East          50.0

✅ Average order amount by customer
  customer  avg_amount
0     Evan       180.0
1    Alice       165.0
2      Bob        75.5
3    Diana        50.0

✅ Top 3 largest sales amounts
   order_id customer region  amount  amount_with_tax
0         3    Alice  North   210.0            231.0
1         5     Evan   West   180.0            198.0
2         1    Alice  North   120.0            132.0
✅ Finished stage 4/4: Query DuckDB database

🎉 Pipeline completed successfully
(base) Harkirats-MacBook-Pro-3:project1_p1_de-main yoharks$
```